

SC2006 Software Engineering

Lecture 16 Supplement

Conrad Watt

Lt15 brief recap

- Observer pattern (again, again)
- Model-View-Controller (MVC) architecture

Testing - a vocabulary

- **Unit Testing**

- test individual functions/small components of program
- generally simple input-output tests

- **Integration Testing**

- test a subsystem of the program

- **End-to-end testing (system testing)**

- Test the software as a whole against requirements
- if you listed functional requirements at the start of the dev process, this is where they are checked!

Testing - a vocabulary

- **White box testing**

- look at the program code and test in a way that tortures its edge cases

e.g.

```
if (length (my_list) < 10) {  
    sort_with_alg1(my_list);  
} else {  
    sort_with_alg2(my_list);  
}
```

White box testing would guide you to test with lists of length <10, 10, >10

Testing - a vocabulary

- **Black box testing**

- you don't know any implementation details of the software (maybe it's someone else's binary!)
- can only test inputs and expected outputs based on its documentation
- e.g. "I know this library is meant to sort a list, so I'll try inputting lots of different lists!"

Testing - a vocabulary

- White box testing vs black box testing
 - if you're testing code you wrote yourself, or code written by your team, manually-written tests are generally assumed to be white-box
 - “black box testing” as a term is useful to explicitly highlight a lack of knowledge, which may mean important edge-cases in internal logic are missed

Testing - a vocabulary

- **Exhaustive testing?**

What if we could test every possible combination of input and expected output for a given function/component/system?

- test all values

not realistic, especially if input format is complicated

- all equivalence classes (we'll see later)

group like values together and test representatives

Testing - a vocabulary

- Exhaustive testing?
- **code coverage**
measure how much of the codebase is executed across the test suite

choice of granularity:

- function
check every function is called
- statement
check every line of code is executed
- path(?)
check every path through the code under test is executed (often not realistic if there are loops)

Testing - a vocabulary

- **Regression testing**

After fixing a bug, it's sensible to add a test that would have triggered the bug, in order to see if it comes back

- **Acceptance testing**

Test whether the software works well in user scenarios

- alpha / pre-alpha
- beta
- release candidate (gamma)
- A/B testing

give different users different versions of the software

Testing - a vocabulary

- **fuzz testing**

in situations where we can't write exhaustive tests, we can try to catch extra bugs by testing recent commits with random inputs

- **oracle testing**

some system exists to determine the expected output

- **differential testing**

test different versions of the system and look for odd-one-out

- **mutation testing**

start with an input that has a known behaviour and introduce “harmless” input changes that should maintain the behaviour

Testing - a vocabulary

- **symbolic execution**

try to explore multiple program paths at once using some constraint system

- **dynamic (concolic testing)**

simulate an execution of the program which splits into multiple executions at each decision point

- **static (abstract interpretation)**

attend to exhaustively simulate all possible executions of the program at once

- **Verification**

“prove” that the program works as expected using some formal or mathematical model

(my research)

Testing - a vocabulary

- Even verification isn't good enough sometimes!
 - can any program be “proven not to crash” when someone can always kick the computer over?
- **Chaos testing** injects random faults into the program/system under evaluation to test its ***robustness***